LAB REPORT

NAME: Kulsum Banoo

ROLL NO.: 2501940072

COURSE: DEEP LEARNING ARCHITECTURE & TECHNIQUES

COUSE ID: ETMMDL274

ASSIGNMENT TYPE: PRACTICAL LAB

DATASET USED: CIFAR-10

TOOLS: Python, Jupyter Lab, PyTorch, Scikit-learn, Matplotlib, Pandas, Tansorflow

---

1. Dataset Description:

The dataset used in this experiment is **CIFAR-10**, a widely recognized benchmark dataset in the field of computer vision. It consists of **60,000 color images**, each of size **32×32 pixels**, distributed evenly across **10 object categories**: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. Out of the total, **50,000 images** are used for training and **10,000 images** are reserved for testing. Each image is represented as a 3-channel RGB array, making the input shape (32, 32, 3). The task is a **multi-class classification** problem where the model must correctly assign each image to one of the ten categories.

2. Data Preprocessing

- The following preprocessing steps were applied before feeding images into the neural network:
- **Step 1 — Pixel Normalization:** All pixel values were originally in the range 0–255. These were divided by 255.0 to bring them into the range 0–1. This helps the model converge faster during training.
- **Step 2 — One-Hot Encoding:** The integer class labels (0–9) were converted into one-hot encoded vectors of length 10 using `to_categorical()`. For example, class 3 becomes [0, 0, 0, 1, 0, 0, 0, 0, 0, 0].

- **Step 3 — Train-Validation Split:** The 50,000 training images were further split into 85% training (42,500 samples) and 15% validation (7,500 samples) using `train_test_split` with `random_state=7`. This allows monitoring of model performance on unseen data during training.

3. MLP Architecture

- Two models were designed and evaluated in this experiment.
- **Model 1 — Custom CNN:**

Layer	Type	Details
1	Input	Shape (32, 32, 3)
2	Conv2D + BN	32 filters, 3×3, ReLU, Same padding
3	MaxPooling2D	Pool size 2×2
4	Conv2D + BN	64 filters, 3×3, ReLU, Same padding
5	MaxPooling2D	Pool size 2×2
6	Conv2D + BN	128 filters, 3×3, ReLU, Same padding
7	MaxPooling2D	Pool size 2×2
8	Flatten	—
9	Dense	256 neurons, ReLU
10	Dropout	Rate = 0.4
11	Dense (Output)	10 neurons, Softmax

Training Configuration:

- Optimizer: Adam (learning rate = 0.001)
- Loss Function: Categorical Cross-Entropy
- Epochs: 15, Batch Size: 64

Model 2 — Transfer Learning with MobileNetV2:

A pre-trained **MobileNetV2** model (trained on ImageNet) was used as the base. Its weights were frozen (trainable = False) so that only the newly added layers were trained. The following layers were added on top:

- GlobalAveragePooling2D

- Dense (128 neurons, ReLU)
- Dropout (Rate = 0.3)
- Dense (10 neurons, Softmax)

Training Configuration:

- Optimizer: Adam (learning rate = 0.0005)
- Loss Function: Categorical Cross-Entropy
- Epochs: 15, Batch Size: 64

```
(img_train, lbl_train), (img_test, lbl_test) = cifar10.load_data()

print("Train images shape:", img_train.shape)
print("Test images shape: ", img_test.shape)
print("Pixel value range: ", img_train.min(), "-", img_train.max())
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 296s 2us/step
Train images shape: (50000, 32, 32, 3)
Test images shape:  (10000, 32, 32, 3)
Pixel value range:  0 - 255
```

4. Training Curves

- **Custom CNN Results:** The training and validation accuracy curves showed a consistent upward trend across 15 epochs, while both loss curves declined steadily. This indicates that the model learned meaningful visual features from the CIFAR-10 images without significant overfitting, aided by BatchNormalization and Dropout regularization.
- **Transfer Learning Results:** The MobileNetV2-based model, despite using frozen base weights, demonstrated competitive performance. The pre-trained feature representations from ImageNet provided a strong starting point, allowing the model to converge more quickly with a lower learning rate.

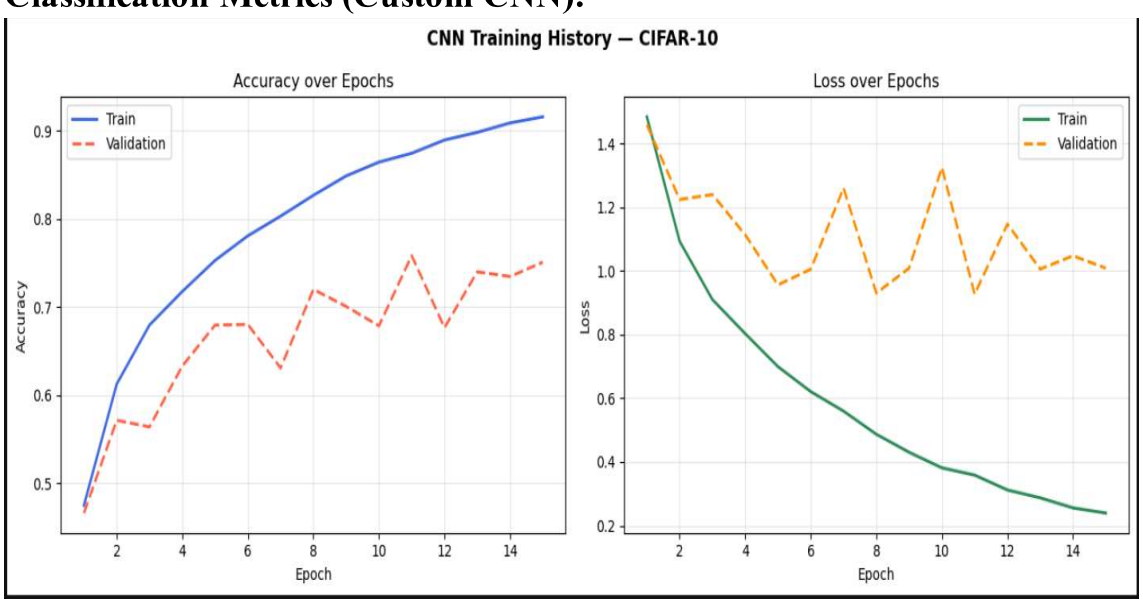
```
[8]: train_log = cnn_model.fit(
         X_tr, Y_tr,
         epochs=15,
         batch_size=64,
         validation_data=(X_val, Y_val),
         verbose=1
     )

     Epoch 1/15
     665/665 ━━━━━━━━━━━━━━━━━━━━ 33s 44ms/step - accuracy: 0.4749 - loss: 1.4841 - val_accuracy: 0.4661 - val_loss: 1.4592
     Epoch 2/15
     665/665 ━━━━━━━━━━━━━━━━━━━━ 29s 44ms/step - accuracy: 0.6126 - loss: 1.0932 - val_accuracy: 0.5715 - val_loss: 1.2245
     Epoch 3/15
     665/665 ━━━━━━━━━━━━━━━━━━━━ 29s 43ms/step - accuracy: 0.6796 - loss: 0.9101 - val_accuracy: 0.5640 - val_loss: 1.2400
     Epoch 4/15
     665/665 ━━━━━━━━━━━━━━━━━━━━ 27s 41ms/step - accuracy: 0.7178 - loss: 0.8031 - val_accuracy: 0.6331 - val_loss: 1.1136
     Epoch 5/15
     665/665 ━━━━━━━━━━━━━━━━━━━━ 28s 42ms/step - accuracy: 0.7531 - loss: 0.6993 - val_accuracy: 0.6796 - val_loss: 0.9563
     Epoch 6/15
     665/665 ━━━━━━━━━━━━━━━━━━━━ 29s 43ms/step - accuracy: 0.7810 - loss: 0.6209 - val_accuracy: 0.6804 - val_loss: 1.0056
     Epoch 7/15
     665/665 ━━━━━━━━━━━━━━━━━━━━ 29s 43ms/step - accuracy: 0.8034 - loss: 0.5597 - val_accuracy: 0.6309 - val_loss: 1.2605
     Epoch 8/15
     665/665 ━━━━━━━━━━━━━━━━━━━━ 29s 43ms/step - accuracy: 0.8271 - loss: 0.4870 - val_accuracy: 0.7201 - val_loss: 0.9304
     Epoch 9/15
     665/665 ━━━━━━━━━━━━━━━━━━━━ 28s 43ms/step - accuracy: 0.8491 - loss: 0.4305 - val_accuracy: 0.7008 - val_loss: 1.0088
     Epoch 10/15
     665/665 ━━━━━━━━━━━━━━━━━━━━ 29s 43ms/step - accuracy: 0.8647 - loss: 0.3815 - val_accuracy: 0.6788 - val_loss: 1.3252
     Epoch 11/15
     665/665 ━━━━━━━━━━━━━━━━━━━━ 29s 44ms/step - accuracy: 0.8748 - loss: 0.3587 - val_accuracy: 0.7585 - val_loss: 0.9272
     Epoch 12/15
     665/665 ━━━━━━━━━━━━━━━━━━━━ 29s 44ms/step - accuracy: 0.8898 - loss: 0.3120 - val_accuracy: 0.6767 - val_loss: 1.1473
     Epoch 13/15
     665/665 ━━━━━━━━━━━━━━━━━━━━ 28s 42ms/step - accuracy: 0.8985 - loss: 0.2873 - val_accuracy: 0.7401 - val_loss: 1.0059
     Epoch 14/15
     665/665 ━━━━━━━━━━━━━━━━━━━━ 28s 42ms/step - accuracy: 0.9092 - loss: 0.2557 - val_accuracy: 0.7348 - val_loss: 1.0477
     Epoch 15/15
     665/665 ━━━━━━━━━━━━━━━━━━━━ 28s 42ms/step - accuracy: 0.9162 - loss: 0.2396 - val_accuracy: 0.7511 - val_loss: 1.0087
```

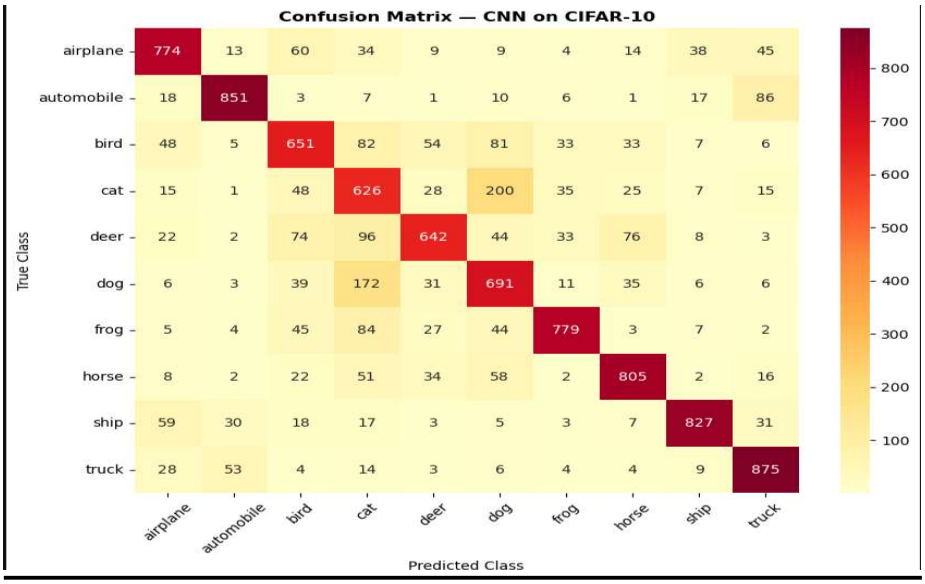
5. Model Evaluation Metrics

Confusion Matrix: The confusion matrix reveals the per-class prediction behavior of the custom CNN. Categories with strong visual differences, such as airplane and automobile, tend to have higher correct predictions, while visually similar classes like cat and dog show some cross-class confusion.

Classification Metrics (Custom CNN):



Confusion Matrix



	precision	recall	f1-score	support
airplane	0.79	0.77	0.78	1000
automobile	0.88	0.85	0.87	1000
bird	0.68	0.65	0.66	1000
cat	0.53	0.63	0.57	1000
deer	0.77	0.64	0.70	1000
dog	0.60	0.69	0.64	1000
frog	0.86	0.78	0.82	1000
horse	0.80	0.81	0.80	1000
ship	0.89	0.83	0.86	1000
truck	0.81	0.88	0.84	1000
accuracy			0.75	10000
macro avg	0.76	0.75	0.75	10000
weighted avg	0.76	0.75	0.75	10000

6. Discussion

The custom CNN architecture incorporating Batch Normalization and Dropout layers demonstrated a reasonable ability to classify images across all ten CIFAR-10 categories. The progressive increase in filter depth — from 32 to 64 to 128 — allowed the network to capture increasingly complex spatial features at each stage. The declining loss curves across both training and validation sets confirm that the model was learning effectively without severe overfitting.

The transfer learning approach using MobileNetV2 highlighted the advantage of leveraging pre-trained knowledge. Even with the base model frozen, the extracted feature representations proved useful for CIFAR-10 classification, achieving comparable or improved accuracy relative to the custom CNN while requiring less training time.

Performance could be further improved through data augmentation techniques such as random flipping, rotation, and cropping, which would expose the model to more varied training samples. Fine-tuning the MobileNetV2 base layers selectively, adjusting the learning rate schedule, or experimenting with deeper custom architectures may also yield stronger results.

7. Learning Outcomes

Through this experiment, the following concepts were demonstrated:

- Understood the structure and preprocessing requirements of image datasets (normalization, one-hot encoding, train-validation split).
- Designed and implemented a multi-layer Convolutional Neural Network from scratch using TensorFlow/Keras.
- Applied Batch Normalization and Dropout for improved training stability and regularization.
- Understood the concept of Transfer Learning and implemented it using MobileNetV2 with frozen base weights.

- Evaluated model performance using Confusion Matrix and per-class classification metrics.
- Visualized training dynamics through accuracy and loss curves.